Triple-Threat Inventory Intelligence System

ML-Powered Inventory Management with Clustering, Forecasting, and Risk Classification

---

Object-Oriented Programming — Project Report

B.Tech / BCA — Academic Year 2025–26

---

deepblue		
Member 1	Aadya Vaid	Python Backend & ML Models
Member 2	Ananya Alagh	Java Swing GUI Frontend
Member 3	Kanusha Sharma	Data Engineering & API Integration

Project Title: *Triple-Threat IMS*

Technologies: Python · Flask · Scikit-Learn · Java Swing · JSON-Simple

Dataset: Kaggle — Inventory Management CSV

Department of Computer Science & Engineering

University of Petroleum and Energy Studies

Contents

1 Objective **3**

1.1 What We Are Working On . . . 3

1.2 Why It Is Important . . . 4

2 Dependencies and Theory **5**

2.1 Software Dependencies . . . 5

2.1.1 Python Backend . . . 5

2.1.2 Java Frontend . . . 5

2.1.3 Dataset . . . 5

2.2 Theoretical Background . . . 6

2.2.1 Object-Oriented Programming (OOP) Concepts . . . 6

2.2.2 Machine Learning Models . . . 6

2.2.3 REST API Architecture . . . 8

2.2.4 Feature Engineering . . . 8

3 Methodology **9**

3.1 System Architecture . . . 9

3.2 Step-by-Step Methodology . . . 9

3.2.1 Step 1: Data Ingestion and Cleaning . . . 9

3.2.2 Step 2: Model Training Pipeline . . . 9

3.2.3 Step 3: Flask API Server . . . 10

3.2.4 Step 4: Java GUI Frontend . . . 10

3.2.5 Step 5: API Communication Layer . . . 11

3.2.6 Step 6: OOP Design Patterns in the Code . . . 11

4 Results **12**

4.1 System Modules and Outputs . . . 12

4.1.1 Dashboard Panel . . . 12

4.1.2 Cluster Panel . . . 12

4.1.3 Forecast Panel . . . 13

4.1.4 Risk Guard Panel . . . 13

4.1.5 Add Product Panel . . . 13

4.1.6 AI Chatbot Panel 14
4.2 How the Work Is Presented 14
4.3 Conclusion 14
4.3.1 Pending Work 15
4.3.2 Future Scope 15

5 Contribution **16**

6 Code **18**
6.1 Repository Structure 18
6.2 Execution Steps 19
6.3 Key Code Snippets 19
6.3.1 OOP Class Hierarchy — `inventory_item.py` 19
6.3.2 K-Means Clustering — `clustering_model.py` 20
6.3.3 Random Forest Risk Classifier — `classification_model.py` . . 21
6.3.4 Java SwingWorker API Call — `DashboardPanel.java` 22
6.4 GitHub Repository 23

References **24**

Chapter 1

Objective

1.1 What We Are Working On

The **Triple-Threat Inventory Intelligence System (IMS)** is a full-stack, machine-learning-powered desktop application designed to automate and intelligentize inventory management for retail, manufacturing, and distribution businesses.

The system brings together three distinct AI models— *clustering*, *demand forecasting*, and *risk classification*—into a seamless pipeline, accessible through an interactive Java Swing GUI that communicates with a Python Flask REST backend.

The system provides the following key capabilities:

- **Product Clustering** — Group products into strategic segments (High-Value/Slow-Moving, Cheap/Fast-Moving, Unpredictable/Critical) using K-Means clustering.
- **Demand Forecasting** — Predict the next 7-day sales demand for each product using Polynomial Regression.
- **Stockout Risk Classification** — Classify every product as LOW, MEDIUM, or HIGH stockout risk using a Random Forest classifier.
- **New Product Prediction** — Instantly predict the risk level and cluster for any newly added product.
- **What-If Analysis** — Simulate how changing supplier lead times would affect overall stockout risk.
- **AI Chatbot** — A rule-based inventory assistant that answers natural-language queries about the inventory state.
- **Interactive Dashboard** — Real-time KPI cards showing total products, inventory value, high-risk counts, and supplier reliability.

1.2 Why It Is Important

Inventory management is one of the most critical operations in any product-based business. Poor inventory decisions result in:

- **Stockouts** — Loss of sales, customer dissatisfaction, and emergency procurement costs.
- **Overstocking** — Tied-up capital, storage costs, and potential product expiry.
- **Supplier Risk** — Unreliable or delayed suppliers cause cascading disruptions.

Traditional inventory systems rely on static reorder-point rules that do not account for demand variability, supplier reliability trends, or seasonal fluctuations. Our system addresses all three dimensions simultaneously using machine learning, making inventory decisions *proactive* rather than *reactive*.

Moreover, the project demonstrates the real-world application of core OOP concepts—**inheritance**, **encapsulation**, **abstraction**, and **polymorphism**—across two programming languages (Python and Java) with a REST API acting as the communication bridge.

Chapter 2

Dependencies and Theory

2.1 Software Dependencies

2.1.1 Python Backend

Library	Version	Purpose
Flask	≥2.0	REST API web server
Flask-CORS	≥3.0	Cross-origin request handling
Pandas	≥1.3	Data loading, cleaning, manipulation
NumPy	≥1.21	Numerical computation
Scikit-Learn	≥1.0	ML models (KMeans, RandomForest, LinearRegression)
Joblib	≥1.0	Model serialization (.pkl files)

2.1.2 Java Frontend

Dependency	Version	Purpose
Java SE (JDK)	≥11	Core Java runtime
Java Swing	Built-in	Desktop GUI framework
java.net.HttpURLConnection	Built-in	HTTP REST calls to Flask
json-simple	1.1.1	JSON parsing (`.jar` included)
Nimbus Look & Feel	Built-in	Modern UI theming

2.1.3 Dataset

Dataset: Inventory Management (Kaggle)
File: `Inventory_Management.csv`

Records: ∼1,000+ products
Key Fields: Product_ID, Product_Name, Unit_Price, Stock_Quantity, Reorder_Level, Inventory_Turnover_Rate, Sales_Volume, Supplier_ID, Date_Received, Last_Order_Date, Category, Status

2.2 Theoretical Background

2.2.1 Object-Oriented Programming (OOP) Concepts

OOP is a programming paradigm centered on organizing software into *objects* that encapsulate both *data (attributes)* and *behavior (methods)*. The four pillars are:

Encapsulation Bundling data and methods within a class, restricting direct access to some components to protect the internal state. In our project, the `InventoryItem` class encapsulates all product attributes (`price`, `current_stock`, etc.) and exposes them through a controlled `to_dict()` method.

Inheritance A mechanism allowing a child class to acquire properties and behaviors of a parent class. In `inventory_item.py`, `PremiumItem` and `FastMovingItem` both inherit from `InventoryItem`, extending it with domain-specific fields such as `warranty_months` and `reorder_batch_size`.

Polymorphism The ability of different objects to respond to the same interface differently. The overridden `to_dict()` in subclasses produces different outputs for `InventoryItem`, `PremiumItem`, and `FastMovingItem`.

Abstraction Hiding complex implementation details and exposing only a clean interface. The `InventoryPipeline` class provides a single `train_all()` call that internally orchestrates three independent ML models. The GUI panels call Flask REST endpoints without knowing anything about the ML internals.

2.2.2 Machine Learning Models

K-Means Clustering

K-Means is an unsupervised learning algorithm that partitions n data points into k clusters by minimizing the *within-cluster sum of squares* (WCSS):

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

where $\boldsymbol{\mu}_i$ is the centroid of cluster S_i.

In this project: Products are clustered on three features— *price*, *lead_time_days*, and *sales_volatility*—into three strategic groups:

- High-Value / Slow-Moving
- Cheap / Fast-Moving
- Unpredictable / Critical

Feature standardisation (zero mean, unit variance) is applied via `StandardScaler` before clustering to prevent scale dominance.

Polynomial Regression (Demand Forecasting)

Polynomial Regression extends Linear Regression by fitting a higher-degree polynomial curve to the training data:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d$$

Degree $d = 2$ (quadratic) is used here. The model is trained on simulated time-series data generated by adding Gaussian noise to each product's `daily_sales` across 10 time steps.

Features used: `day` (ordinal), `is_weekend`, `is_holiday`, `price`, `sales_volatility`.

Output: A 7-day demand forecast per product.

Evaluation Metric: R^2 (coefficient of determination):

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Random Forest Classifier (Stockout Risk)

Random Forest is an ensemble method that constructs T decision trees on bootstrapped subsets of data and aggregates their predictions by majority vote (classification):

$$\hat{y} = \text{mode}\big\{h_t(\mathbf{x})\big\}_{t=1}^{T}$$

Each tree h_t is trained on a random subset of features at each split (feature bagging), reducing variance and improving generalisation.

Configuration: `n_estimators=100`, `class_weight="balanced"` (handles class imbalance), `random_state=42`.

Features used: `current_stock`, `days_since_last_order`, `supplier_reliability`, `lead_time_days`, `sales_volatility`, `price`.

Target: Binary label `is_stockout_risk` (1 if `current_stock` < `Reorder_Level`, else 0).

Evaluation: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.

2.2.3 REST API Architecture

A **REST (Representational State Transfer)** API is a stateless web interface where clients make HTTP requests (`GET`, `POST`) to server endpoints that return JSON-formatted responses.

Table 2.3: Flask API Endpoints

Method	Endpoint	Description
GET	`/api/status`	Server health check
GET	`/api/summary`	Dashboard KPI data
POST	`/api/train`	Retrain all models
GET	`/api/clusters`	Product cluster results
GET	`/api/forecast`	Demand forecast (top 20)
GET	`/api/risk`	Stockout risk per product
GET	`/api/categories`	Category-level aggregation
GET	`/api/status_breakdown`	Status-level aggregation
POST	`/api/predict_new`	Predict risk for new product
POST	`/api/chat`	AI Chatbot query

2.2.4 Feature Engineering

Raw CSV data undergoes several transformations before being fed into ML models:

1. **Lead Time** = `|Date_Received  Last_Order_Date|` (clipped 1–30 days)
2. **Days Since Last Order** = latest date in dataset `Last_Order_Date`
3. **Supplier Reliability** = $\frac{1}{1 + \sigma_{\text{lead_time}}}$ per supplier (clipped 0.5–1.0)
4. **Sales Volatility** = `Inventory_Turnover_Rate / 10.0` (clipped 0.5–8.0)
5. **Weekend Flag** = 1 if last order was on Saturday or Sunday
6. **Stockout Risk Label** = 1 if `Stock_Quantity` < `Reorder_Level`

Chapter 3

Methodology

3.1 System Architecture

The system follows a **two-tier client-server architecture**:

$$\boxed{\textbf{Java Swing GUI}} \xrightarrow{\text{HTTP/JSON}} \boxed{\textbf{Python Flask API}}$$

(Frontend) (Backend + ML Models)

The Java frontend sends HTTP requests to `http://localhost:5000` and renders the JSON responses in Swing panels. The Python backend loads a CSV dataset, engineers features, trains three ML models, and serves all results through REST endpoints.

3.2 Step-by-Step Methodology

3.2.1 Step 1: Data Ingestion and Cleaning

The `DataLoader` module (`data_loader.py`) performs:

1. Remove the `$` sign from `Unit_Price` and cast to float.
2. Drop the single null row in `Category`.
3. Parse `Date_Received`, `Last_Order_Date`, and `Expiration_Date` using `pd.to_datetime`.
4. Engineer all derived features (see Section 2.5).
5. Rename columns to match model expectations.

3.2.2 Step 2: Model Training Pipeline

`InventoryPipeline.train_all()` runs three phases sequentially:

1. **Phase 1 — Clustering:** `InventoryClusterer.train(df)` standardises features, fits K-Means, auto-labels clusters from centroid ranks, and saves `kmeans_model.pkl` and `cluster_scaler.pkl`.
2. **Phase 2 — Forecasting:** `DemandForecaster.train(df)` generates 10 simulated time steps per product, fits a polynomial regression, then forecasts the next 7 days for every product, saving `regression_model.pkl`.
3. **Phase 3 — Classification:** `StockoutRiskClassifier.train(df)` trains a Random Forest with an 80/20 train/test split, evaluates accuracy and feature importance, and saves `rf_classifier.pkl`.

Models are saved to the `models/` directory using `joblib`. On startup, the API checks whether `kmeans_model.pkl` exists; if not, it triggers `train_all()` automatically.

3.2.3 Step 3: Flask API Server

`api_server.py` initialises Flask, enables CORS, loads the pipeline on startup, and registers all REST routes. Each endpoint either calls a pipeline method or performs a direct Pandas aggregation.

3.2.4 Step 4: Java GUI Frontend

MainFrame.java is the application entry point. It:

- Sets the Nimbus Look & Feel for modern styling.
- Builds a header panel with the application title and a dark/light mode toggle.
- Creates a `JTabbedPane` with six coloured tabs.

Each tab is a separate Swing panel:

Table 3.1: GUI Panels and Their Functions

Panel	Class	Function
Dashboard	`DashboardPanel`	KPI cards + model retraining
Clusters	`ClusterPanel`	Product clustering results
Forecast	`ForecastPanel`	7-day demand forecast table
Risk Guard	`RiskPanel`	Stockout risk + What-If slider
Add Product	`AddProductPanel`	Form to predict new product
AI Chatbot	`ChatBotPanel`	Natural-language Q&A

All API calls are made asynchronously using `SwingWorker<Void, Void>` to prevent UI freezing.

3.2.5 Step 5: API Communication Layer

`ApiClient.java` wraps `HttpURLConnection` to provide:

- `getRaw(endpoint)` — makes a GET request and returns the raw JSON string.
- `postTrain()` — sends a POST to `/api/train`.

JSON strings are parsed client-side using the `json-simple-1.1.1.jar` library.

3.2.6 Step 6: OOP Design Patterns in the Code

Inheritance (Python): `PremiumItem` and `FastMovingItem` extend `InventoryItem`. Both override `to_dict()` to add specialised fields.

Encapsulation (Python & Java): Each ML model class exposes only `train()`, `predict()`, and `load()` methods. Internal state (model weights, scalers) is hidden.

Abstraction (Python): `InventoryPipeline` abstracts all three model operations behind `train_all()` and `get_summary()`.

Composition (Java): `MainFrame` composes `DashboardPanel`, `ClusterPanel`, `RiskPanel`, etc., rather than inheriting from them, following the OOP principle of *composition over inheritance*.

Chapter 4

Results

4.1 System Modules and Outputs

4.1.1 Dashboard Panel

The Dashboard is the first tab displayed on launch. It fetches data from `/api/summary` and renders six KPI metric cards:

- **Total Products** — count of unique `product_id` values.
- **Total Stock Value** — sum of (`current_stock` × `price`) across all products.
- **High Risk Items** — count of products flagged as stockout risk.
- **Avg Supplier Reliability** — mean of the engineered `supplier_reliability` score.
- **Active Products** — products with `status = "Active"`.
- **Backordered Products** — products with `status = "Backordered"`.

A *System Log* area at the bottom tracks all user actions and API calls in real time.

A **Retrain Models** button triggers a POST to `/api/train`, which retrains all three models and returns the updated accuracy score.

4.1.2 Cluster Panel

The Cluster tab fetches data from `/api/clusters` and displays each product alongside its assigned cluster label. The three semantic clusters are:

1. **High-Value / Slow-Moving** — products with the highest price rank among centroids.

2. **Cheap / Fast-Moving** — low-price, moderate-volatility products.
3. **Unpredictable / Critical** — highest sales volatility rank; require special attention.

Auto-labelling is done by comparing centroid ranks along the `price` and `sales_volatility` dimensions.

4.1.3 Forecast Panel

The Forecast tab fetches results from `/api/forecast` (top 20 products). For each product, it shows:

- Current daily sales figure.
- Predicted sales for each of the next 7 days.
- Total predicted weekly demand.
- The R^2 score of the regression model.

4.1.4 Risk Guard Panel

The Risk Guard tab provides two views:

1. **Standard Risk View** — calls `/api/risk` and displays each product's risk probability and LOW / MEDIUM / HIGH classification.
2. **What-If Analysis** — a slider/input lets the user specify a hypothetical supplier delay (in days). The backend calls `what_if_analysis(df, delay)`, which overrides `lead_time_days` with the new value and re-runs the classifier, showing how risk profiles change.

4.1.5 Add Product Panel

A form with input fields allows the user to enter a new product's details: `name`, `price`, `current_stock`, `lead_time_days`, `days_since_last_order`, `supplier_reliability`, `sales_volatility`, `reorder_level`, `daily_sales`, `category`.

On submission, the form POSTs to `/api/predict_new`. The backend runs both the Random Forest classifier and K-Means on the new product and returns:

- Risk probability (%)
- Risk level (LOW / MEDIUM / HIGH)

- Cluster ID and label
- Stockout flag
- A human-readable recommendation message

4.1.6 AI Chatbot Panel

The Chatbot tab sends user queries to `/api/chat`. The backend implements a keyword-matching engine that answers questions such as:

- "How many products are at risk?"
- "What is the total stock value?"
- "Which products should I reorder?"
- "What is the best/worst category?"
- "Give me a summary"

All answers are computed live from the loaded DataFrame, ensuring up-to-date responses.

4.2 How the Work Is Presented

GUI Presentation: The Java Swing application launches with `MainFrame.main()`, which invokes `SwingUtilities.invokeLater` to start the GUI on the Event Dispatch Thread. All six tabs are immediately accessible. The dark/light mode toggle applies a deep navy or soft blue colour theme across the entire application.

Backend Presentation: The Flask server is started from the terminal using `python api_server.py`, which begins listening on `http://localhost:5000`. Models are auto-trained on the first run if `.pkl` files are absent.

Architecture Presentation: The project follows a clean separation of concerns: data loading, model training, API serving, and GUI rendering are each handled by dedicated classes.

4.3 Conclusion

The Triple-Threat IMS successfully demonstrates the integration of advanced machine learning with robust OOP design:

- All three ML models (K-Means, Polynomial Regression, Random Forest) are trained, serialised, and served through a Flask REST API.
- The Java Swing GUI provides an intuitive, colour-coded interface with real-time data refresh.
- OOP principles are clearly observed in both Python (inheritance, encapsulation) and Java (composition, abstraction).

4.3.1 Pending Work

- User authentication and role-based access (admin vs. viewer).
- Database backend (PostgreSQL / SQLite) to replace CSV.
- Export reports to PDF or Excel from the GUI.
- Unit tests for each pipeline component.
- Live supply-chain data integration via supplier APIs.

4.3.2 Future Scope

- **Deep Learning:** Replace Polynomial Regression with an LSTM model for more accurate time-series demand forecasting.
- **Web Frontend:** Port the GUI to a React.js web app for browser-based access.
- **IoT Integration:** Connect to RFID/barcode scanners for real-time stock updates.
- **Multi-warehouse Support:** Extend the system to manage inventory across multiple locations.
- **NLP Chatbot:** Replace the keyword-matching chatbot with a large language model (LLM)-powered assistant.

Chapter 5

Contribution

Table 5.1: Member Contributions

Member	Area	Detailed Contribution
Member 1	Python Backend & ML Models	Designed and implemented `classification_model.py` (Random Forest stockout risk classifier), `regression_model.py` (Polynomial Regression demand forecaster), and `clustering_model.py` (K-Means clustering). Integrated all three into `pipeline.py`. Handled model serialisation with `joblib`.
Member 2	Java Swing GUI Frontend	Built the entire Java GUI: `MainFrame.java`, `DashboardPanel.java`, `ClusterPanel.java`, `ForecastPanel.java`, `RiskPanel.java`, `AddProductPanel.java`, and `ChatBotPanel.java`. Implemented dark/light mode theming, async `SwingWorker` calls, and JSON parsing using `json-simple`.
Member 3	Data Engineering & API Integration	Developed `data_loader.py` (data cleaning, feature engineering, label creation), `inventory_item.py` (OOP class hierarchy with inheritance), and `api_server.py` (all 10 Flask REST endpoints). Also wrote `ApiClient.java` for HTTP communication. Sourced and validated the Kaggle dataset.

Chapter 6

Code

6.1 Repository Structure

```
TripleThreat_IMS/
 python_backend/
    api_server.py          # Flask REST API
    pipeline.py            # Master ML pipeline
    classification_model.py  # Random Forest
    clustering_model.py      # K-Means
    regression_model.py      # Polynomial Regression
    data_loader.py           # Data cleaning + engineering
    inventory_item.py        # OOP class hierarchy
    data/
       Inventory_Management.csv
    models/                  # Serialised .pkl files
        kmeans_model.pkl
        cluster_scaler.pkl
        regression_model.pkl
        rf_classifier.pkl
 java_gui/
     src/gui/
        MainFrame.java
        DashboardPanel.java
        ClusterPanel.java
        ForecastPanel.java
        RiskPanel.java
        AddProductPanel.java
        ChatBotPanel.java
        ApiClient.java
```

```
lib/
   json-simple-1.1.1.jar
out/gui/                # Compiled .class files
```

6.2 Execution Steps

Step 1 — Start the Python Backend:

```
cd TripleThreat_IMS/python_backend
pip install flask flask-cors pandas scikit-learn joblib
python api_server.py
```

Server starts at `http://localhost:5000`. Models are auto-trained on first run.

Step 2 — Run the Java GUI:

```
cd TripleThreat_IMS/java_gui
javac -cp lib/json-simple-1.1.1.jar -d out src/gui/*.java
java -cp out:lib/json-simple-1.1.1.jar gui.MainFrame
# Windows: use semicolon separator
java -cp "out;lib/json-simple-1.1.1.jar" gui.MainFrame
```

6.3 Key Code Snippets

6.3.1 OOP Class Hierarchy — inventory_item.py

Listing 6.1: Inheritance in InventoryItem

```python
class InventoryItem:
    def __init__(self, product_id, name, price, current_stock,
                 lead_time_days, sales_volatility,
                    supplier_reliability):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.current_stock = current_stock
        self.lead_time_days = lead_time_days
        self.sales_volatility = sales_volatility
        self.supplier_reliability = supplier_reliability

    def to_dict(self):
```

```
        return {
            "product_id": self.product_id,
            "name": self.name,
            "price": self.price,
            "current_stock": self.current_stock,
            "lead_time_days": self.lead_time_days,
            "sales_volatility": self.sales_volatility,
            "supplier_reliability": self.supplier_reliability
        }

class PremiumItem(InventoryItem):        # Inheritance
    def __init__(self, *args, warranty_months=12):
        super().__init__(*args)           # Call parent constructor
        self.warranty_months = warranty_months

    def to_dict(self):                   # Method overriding
        d = super().to_dict()
        d["warranty_months"] = self.warranty_months
        return d
```

6.3.2 K-Means Clustering — clustering_model.py

Listing 6.2: KMeans Training and Auto-Labelling

```
class InventoryClusterer:
    def __init__(self, n_clusters=3):
        self.scaler = StandardScaler()
        self.model  = KMeans(n_clusters=n_clusters,
                             random_state=42, n_init=10)
        self.feature_cols = ["price","lead_time_days","sales_volatility"]

    def train(self, df):
        product_df = df[["product_id","name"] +
                        self.feature_cols].drop_duplicates("product_id")
        X = self.scaler.fit_transform(product_df[self.feature_cols])
        product_df["cluster"] = self.model.fit_predict(X)

        centroids = pd.DataFrame(
```

```
            self.scaler.inverse_transform(self.model.cluster_centers_),
            columns=self.feature_cols)

        label_map = {}
        for i in range(len(centroids)):
            if centroids["price"].rank()[i] == len(centroids):
                label_map[i] = "High-Value / Slow-Moving"
            elif centroids["sales_volatility"].rank()[i] == len(centroids):
                label_map[i] = "Unpredictable / Critical"
            else:
                label_map[i] = "Cheap / Fast-Moving"

        product_df["cluster_label"] = product_df["cluster"].map(label_map)
        joblib.dump(self.model,  "models/kmeans_model.pkl")
        joblib.dump(self.scaler, "models/cluster_scaler.pkl")
        return product_df.to_dict(orient="records")
```

6.3.3 Random Forest Risk Classifier — classification_model.py

Listing 6.3: Random Forest Training and What-If Analysis

```
class StockoutRiskClassifier:
    def __init__(self):
        self.model = RandomForestClassifier(
            n_estimators=100, random_state=42, class_weight="balanced")
        self.feature_cols = ["current_stock","days_since_last_order",
                             "supplier_reliability","lead_time_days",
                             "sales_volatility","price"]

    def train(self, df):
        product_df = df.drop_duplicates("product_id").reset_index(drop=True)
        X = product_df[self.feature_cols]
        y = product_df["is_stockout_risk"]
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42)
```

```python
        self.model.fit(X_train, y_train)
        report = classification_report(y_test, self.model.predict(X_test),
                                       output_dict=True, zero_division=0)
        joblib.dump(self.model, "models/rf_classifier.pkl")
        return {"accuracy": round(report["accuracy"], 3)}

    def what_if_analysis(self, df, supplier_delay_days):
        """Simulate impact of changed supplier lead time."""
        self.model = joblib.load("models/rf_classifier.pkl")
        product_df = df.drop_duplicates("product_id").copy()
        product_df["lead_time_days"] = supplier_delay_days  # override
        X = product_df[self.feature_cols]
        probs = self.model.predict_proba(X)[:, 1]
        product_df["risk_probability"] = probs
        product_df["risk_level"] = pd.cut(
            probs, bins=[0,0.4,0.7,1.01], labels=["LOW","MEDIUM","HIGH"])
        return product_df[["product_id","name","category",
                           "current_stock","risk_probability",
                           "risk_level"]].to_dict(orient="records")
```

6.3.4 Java SwingWorker API Call — DashboardPanel.java

Listing 6.4: Asynchronous API Call using SwingWorker

```java
public void refresh() {
    log("Refreshing dashboard...");
    new SwingWorker<Void, Void>() {
        protected Void doInBackground() throws Exception {
            String raw = ApiClient.getRaw("/summary");
            JSONObject data = (JSONObject) new JSONParser().parse(raw);
            SwingUtilities.invokeLater(() -> {
                totalProductsLabel.setText(
                    data.get("total_products").toString());
                stockValueLabel.setText(
                    "$" + data.get("total_stock_value").toString());
```

```
                highRiskLabel.setText(
                    data.get("high_risk_count").toString());
                log("Dashboard refreshed successfully.");
            });
            return null;
        }
        protected void done() {
            try { get(); }
            catch (Exception e) { log("Error: " + e.getMessage
                ()); }
        }
    }.execute();
}
```

6.4 GitHub Repository

All source code, the compiled `.class` files, the dataset, and pre-trained model `.pkl` files are maintained in the project GitHub repository:

Repository URL: https://github.com/[YourUsername]/TripleThreat_IMS

How to clone:

```
git clone https://github.com/[YourUsername]/TripleThreat_IMS.git
```

Replace `[YourUsername]` with your actual GitHub username before submission.

References

1. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
2. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
4. Flask Documentation. (2024). https://flask.palletsprojects.com/
5. Oracle Java Swing Tutorial. https://docs.oracle.com/javase/tutorial/uiswing/
6. Kaggle Dataset: Inventory Management. https://www.kaggle.com/
7. Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.
8. Grus, J. (2019). *Data Science from Scratch* (2nd ed.). O'Reilly Media.